

```
with oqs.Signature('Dilithium2') as signer:
    public_key = signer.generate_keypair()
    signature = signer.sign(msg)
```

Figure 1: Client-side code

```
# Verify the signature
with oqs.Signature('Dilithium2') as verifier:
    if verifier.verify(message, signature, public_key):
        print(f"[{addr}] Verified: {message.decode()}")
    else:
        print(f"[{addr}] Invalid signature!")
```

Figure 2: Server-side code

```
anisha@Anisha: /mnt/d/AG_DOC$ python3 server.py
/usr/local/lib/python3.12/dist-packages/oqs/__init__.py:1: UserWarning: liboqs version (major, minor) 0.13.1-dev differs from liboqs-python version 0.12.0
from oqs.oqs import (
[Server] Listening on port 65432
[Server] Connected by ('127.0.0.1', 55234)
[('127.0.0.1', 55234)] Verified: Hi
[('127.0.0.1', 55234)] Verified: This is an encrypted message
[('127.0.0.1', 55234)] Verified: This is using PQC standards

anisha@Anisha: /mnt/d/AG_DOC$ python3 client.py
/usr/local/lib/python3.12/dist-packages/oqs/__init__.py:1: UserWarning: liboqs version (major, minor) 0.13.1-dev differs from liboqs-python version 0.12.0
from oqs.oqs import (
[Client] Connected to server. Type messages to send.
You: Hi
You: This is an encrypted message
You: This is using PQC standards
You:
```

Figure 3: Client signature verification

Several cryptographic libraries already support Dilithium, including BoringSSL, OpenSSL, WolfSSL, Libcrypt, and IBM's PKCS#11. For our implementation, we make use of liboqs, a free and open source C library from the Open Quantum Safe (OQS) project, available on GitHub. This library provides a broad range of quantum-safe cryptographic algorithms, making it well-suited for both experimental and production environments.

The liboqs library offers three major capabilities:

- A collection of open source implementations of quantum-safe key encapsulation mechanisms (KEMs) and digital signature algorithms (see its supported algorithms list).
- A common API for all supported algorithms.
- A test harness and benchmarking

routines for evaluation and performance measurement.

For communication between the client and server, we use Python with socket programming. The connection is established through an assigned port (e.g., 65432) and a host IP address (e.g., 127.0.0.1 for localhost, though any valid IP can be used).

The client generates a public key using the Dilithium library and signs the message with its private key, as shown in Figure 1.

The client then transmits its public key and the signed message to the server through the same socket connection. On the server side, the socket is bound to the specified address and port to receive the client's signed message. Using the OQS library, the server verifies the integrity and authenticity of the communication by validating the signature with the client's public key and comparing it with the original

message. The server-side code is shown in Figure 2.

After successful verification, the command and output shown in Figure 3 demonstrates the client-server communication flow. It is important to encode messages into bytes—either using the pickle library or through packet encapsulation—to ensure seamless data exchange over the socket interface.

One of the key advantages of CRYSTALS-Dilithium is its strong balance between performance and cryptographic strength. It is both efficient and compact when compared

to other post-quantum digital signature algorithms.

In our implementation, we utilise Dilithium2 (128-bit security level), which features the following parameter sizes:

- *Signature size*: ~2.4KB
- *Public key size*: ~1.3KB
- *Private key size*: ~2.5KB

When compared to SPHINCS+, another post-quantum signature scheme, Dilithium provides much faster signing and verification speeds while also producing significantly smaller signatures. Although traditional ECDSA signatures are shorter (typically 64-72 bytes), they lack quantum resistance, rendering them unsuitable for the post-quantum era. **END** 🐙

 **By: Anisha Ghosh**

The author is a cybersecurity researcher at the University of Tokyo and actively contributes to open source projects.